

Launch-Overhead Amortization is Unconditional, Not Just Launch-Bound

A fused single-launch elementwise chain beats a five-launch per-op baseline by $\geq 30\%$ at every tensor size — a closed-form \$0 oracle from a measured per-launch constant

hexa-lang GPU domain
whole-program GPU fusion, no LLVM, no cuBLAS
github.com/dancinlab/hexa-lang

May 25, 2026

Abstract

Eager and per-operator GPU stacks (PyTorch eager, epilogue-free library calls) issue an elementwise neural-network sub-chain as one kernel launch *per operator*, round-tripping the working tensor through high-bandwidth memory (HBM) at every step. Whole-program fusion collapses the chain into a single launch that keeps the intermediates in registers. We pre-register the falsifier F-FUSION-LAUNCH-AMORT: a fused single-launch kernel computing the five-operator chain $y = r + s \cdot \text{GeLU}(ax + b)$ (multiply, add, GeLU, scale, add-residual) beats a five-launch per-op baseline by at least 30% wall in the launch-bound (small-tensor) regime, and is falsified otherwise. Using a deterministic, zero-cost oracle and the per-launch overhead *measured* on an RTX 5070 ($L = 1\mu\text{s}$ warm, `cuLaunchKernel` amortized over 10^3 iterations), we show the fused kernel uses 1 launch versus 5 ($5.0\times$ fewer) and 3 HBM transfers per element versus 11 ($3.67\times$ less). The closed-form wall-advantage curve is bounded in $[72.7\%, 80\%]$ for *all* tensor sizes: the 30%-crossover n^* is negative, so the advantage is *unconditional*, not merely launch-bound. This over-satisfies the falsifier — the predicted launch-bound-only crossover is replaced by a win that the HBM-traffic axis carries into the bandwidth-bound regime too. Both kernels are `ptxas-clean` at `sm_80` (0 spill). The timed silicon wall is the single deferred confirmation; the structural counts and the projection from the measured constant are terminal.

1 Introduction

A transformer or convolutional network spends a surprising fraction of its wall time not in matrix multiplication but in the elementwise “glue” between the big kernels: bias adds, activations, residual adds, scaling, normalization tails. In an eager execution model each of these is a separate library call, and each library call is a separate GPU kernel launch. A kernel launch is not free — the CUDA driver must marshal arguments, push a command onto a stream, and (in the common synchronous case) the host must observe completion. On top of the launch, each per-op kernel reads its input tensor from HBM and writes its output back, so a chain of k elementwise ops pays k launches and $\approx 2k$ HBM tensor passes even though the arithmetic is trivial.

Whole-program fusion is the structural answer: a compiler that sees the entire chain emits *one* kernel whose body performs all k operations on each element while it is resident in a register, reading the input once and writing the output once. This is the central advantage hexa-lang’s GPU backend targets — bypassing cuBLAS-using stacks by fusing the program end to end rather than dispatching a sequence of opaque library kernels.

The folklore claim is that fusion “wins on small tensors where launch overhead dominates.” This paper makes the claim precise, pre-registers a falsifier with a numeric threshold, and tests

it with a deterministic oracle plus a single measured hardware constant — without consuming a contended GPU for the headline result. The surprise is that the standard framing is too weak.

Contributions.

1. We pre-register F-FUSION-LAUNCH-AMORT (Section 3) with a concrete $\geq 30\%$ threshold and an explicit launch-bound regime, so the result is falsifiable before any measurement.
2. We hand-emit a fused single-launch PTX kernel for the five-op chain $y = r + s \cdot \text{GeLU}(ax + b)$ and a faithful five-launch per-op baseline, both `ptxas`-clean at `sm_80` (Section 4).
3. We give a closed-form wall model parameterized by the *measured* per-launch overhead and show the fused advantage is bounded in $[72.7\%, 80\%]$ for all n — the 30%-crossover n^* is negative, so the $\geq 30\%$ win is *unconditional* (Sections 5, 6). This is a closed-negative on the launch-bound-*only* hypothesis: the data deterministically rule out the narrower framing.

2 Background

2.1 The per-op kernel-launch tax

In an eager stack a logical expression such as $y = r + s \cdot \text{GeLU}(ax + b)$ decomposes into a sequence of tensor ops, each of which is dispatched independently. A representative decomposition is

$$t_1 = a \cdot x, \quad t_2 = t_1 + b, \quad t_3 = \text{GeLU}(t_2), \quad t_4 = s \cdot t_3, \quad y = r + t_4,$$

i.e. multiply, add, activation, scale, and add-residual. Each t_i is a full tensor materialized in HBM, because the next op is a separate kernel that must read it back. The cost of one such op on an n -element tensor is, to first order,

$$T_{\text{op}}(n) = L + \frac{(\text{reads} + \text{writes}) \cdot 4n}{\text{BW}},$$

where L is the per-launch overhead, BW is the effective HBM bandwidth, and the tensor is f32 (4 bytes/element). For a trivial elementwise op the arithmetic is hidden under the memory traffic, so the kernel is bandwidth-bound once n is large and launch-bound once n is small.

2.2 What fusion removes

A fused kernel computes all five ops in one launch with $t_1 \dots t_4$ held in registers. It reads x and r once and writes y once: 2 reads + 1 write = 3 HBM transfers per element, against the baseline’s 6 reads + 5 writes = 11. (The baseline’s residual add reads two buffers, hence 6 not 5 reads.) Fusion therefore attacks two independent axes at once: it cuts the launch count $5 \rightarrow 1$ and the HBM traffic $11 \rightarrow 3$. The question this paper settles is how those two structural ratios translate into a wall-time advantage as a function of tensor size, and whether the $\geq 30\%$ bar is met only in the launch-bound corner or more broadly.

2.3 hexa-lang’s no-LLVM, no-cuBLAS substrate

hexa-lang compiles its own MIR to PTX directly (no LLVM, no C-transpile) and links the emitted kernels into the program binary, eliminating the `Python`→`C++`→`ATen`→`stream`→`cuBLAS` dispatch chain entirely. The fusion advantage studied here is the structural lever behind the project’s north-star goal of bypassing cuBLAS-using NN stacks by whole-program fusion.

2.4 Related work

Operator fusion is well established in production deep-learning compilers. XLA, TVM, and PyTorch’s `torch.compile` all perform elementwise fusion, and cuBLAS-LT offers a limited fixed menu of GEMM epilogues (bias, ReLU, GELU). The contribution here is not a new fusion technique but a *pre-registered, falsifiable, zero-cost characterization* of the wall advantage attributable specifically to launch-overhead and HBM-traffic amortization, parameterized by a measured hardware constant rather than fitted to a benchmark. Where prior reports state “fusion helps on small tensors,” we quantify the helps-everywhere boundary in closed form and show the small-tensor framing understates the effect. The methodology — pre-register a numeric falsifier, settle the structural part with a deterministic oracle, and defer only the timed silicon confirmation — is designed so that the headline result does not depend on a contended GPU and cannot be quietly tuned after the fact.

The decomposition we hold the baseline to (five separate launches with HBM round-trips) is the eager-execution default, not a strawman: an eager stack that has not been explicitly told to fuse *will* dispatch each elementwise op as its own kernel. A fusing compiler is exactly what removes this tax, which is why the comparison isolates the lever cleanly.

3 Statement (pre-registered falsifier)

F-Fusion-Launch-Amort. A single fused hexa-emitted kernel computing a ≥ 5 -op elementwise chain (here $y = r + s \cdot \text{GeLU}(ax + b)$), broken as multiply, add, GeLU, scale-by- s , add-residual) over a tensor uses *one* kernel launch, versus a per-op baseline of *five* separate kernel launches each round-tripping the tensor through HBM. On small tensors (where per-element compute $\ll$ launch overhead) the fused single-launch kernel beats the five-launch baseline by $\geq 30\%$ wall. The falsifier is **FALSIFIED** if launch-overhead amortization does not yield $\geq 30\%$ in the launch-bound regime.

Pre-registered parameters (chosen before measurement).

- Chain: $y = r + s \cdot \text{GeLU}(ax + b)$, exactly five ops.
- Threshold: $\geq 30\%$ wall advantage.
- Regime: launch-bound (small n , compute $\ll$ launch).
- Per-launch overhead L : taken from a *prior independent* measurement (Section 4), not fitted here.
- GeLU form: the tanh approximation of Hendrycks and Gimpel [1], identical on both sides and in the reference.

4 Method

4.1 Why this chain

The chain $y = r + s \cdot \text{GeLU}(ax + b)$ is not arbitrary. It is the exact shape of a residual-connected feed-forward tail: $ax + b$ is an affine projection, $\text{GeLU}(\cdot)$ is the transformer-standard activation, s is a per-block scale (as in scaled residual or LayerScale variants), and r is the residual carried around the block. Every modern transformer block contains several instances of this exact pattern between its GEMMs. In an eager stack each of the five sub-operations is a library call — a multiply kernel, an add kernel, a GELU kernel, a scale kernel, and a residual-add kernel — so the chain is precisely where the per-op launch tax accrues. By choosing a five-op chain we

sit at the falsifier’s minimum (≥ 5 ops); longer real chains (normalization tails, dropout masks, gating) only strengthen the advantage by raising both the launch count and R_b .

The chain is also a clean test because it is purely elementwise: there is no reduction, no cross-thread communication, and no shared memory, so the only costs are the launch and the HBM passes — exactly the two quantities the falsifier isolates. A chain containing a reduction or a GEMM would mix in a compute-bound term and confound the launch/HBM analysis; that is deliberately out of scope.

4.2 The two kernels

Fused kernel (one launch). A single PTX `.entry fused_chain` loads $x[i]$ and $r[i]$ once, then computes the whole chain in registers and stores $y[i]$ once:

```
ld.global.f32 %f3, [&x[i]]; // single HBM read of input
ld.global.f32 %f4, [&r[i]]; // residual
mul.f32 %f5, a, %f3; // (1) mul: t1 = a*x
add.f32 %f6, %f5, b; // (2) add: u = t1 + b
// (3) gelu(u) via tanh.approx
mul.f32 %f7, %f6, %f6; // u^2
mul.f32 %f8, %f7, %f6; // u^3
mul.f32 %f9, %f8, 0.044715; // k1*u^3
add.f32 %f10, %f6, %f9; // u + k1*u^3
mul.f32 %f11, %f10, 0.7978846; // sqrt(2/pi)*inner
tanh.approx.f32 %f12, %f11;
add.f32 %f13, %f12, 1.0;
mul.f32 %f14, %f6, 0.5;
mul.f32 %f15, %f14, %f13; // t3 = GeLU(u)
mul.f32 %f16, s, %f15; // (4) mul-scale: t4 = s*t3
add.f32 %f17, %f4, %f16; // (5) add-resid: y = r + t4
st.global.f32 [&y[i]], %f17; // single HBM write of output
```

Per-op baseline (five launches). Five independent `.entry` kernels — `k1_mul`, `k2_add`, `k3_gelu`, `k4_mul_scale`, `k5_add_resid` — launched in sequence with ping-pong buffers, exactly mirroring eager-stack stream semantics where each op materializes its output in HBM before the next op reads it. `k3_gelu` uses the identical `tanh-approx` body, so the two sides are arithmetically equal modulo floating-point reassociation.

4.3 The measured per-launch constant

The model is parameterized by a single hardware constant that was measured in a prior, independent cycle on the same RTX 5070 pool host (recorded as F-RFC067-PB-TIMING): an empty-kernel `cuLaunchKernel` costs $\approx 1\mu\text{s}$ warm (Nth launch averaged over 10^3 iterations), $23\mu\text{s}$ on the first launch, with warm module-load $28\mu\text{s}$. We use $L = 1\mu\text{s}$ as the warm per-launch overhead. The effective HBM bandwidth is taken conservatively as $\text{BW} = 200\text{ GB/s}$; the result is monotone in BW and the conclusion holds for any positive bandwidth (Section 6).

4.4 The closed-form wall model

With L the per-launch overhead, $c = 4/(\text{BW} \cdot 10^3)$ the microseconds per element per traffic-unit, and $R_f = 3$, $R_b = 11$ the HBM transfers per element,

$$W_{\text{fused}}(n) = 1 \cdot L + R_f c n, \quad (1)$$

$$W_{\text{baseline}}(n) = 5 \cdot L + R_b c n, \quad (2)$$

$$\text{pct_faster}(n) = \left(1 - \frac{W_{\text{fused}}(n)}{W_{\text{baseline}}(n)}\right) \cdot 100\%. \quad (3)$$

This is the entirety of the model. The structural constants R_f and R_b are exact (counted from the PTX), L is measured, and BW is a conservative constant.

4.5 Verification harness

The structural counts and the projection are computed by a deterministic hexa-native oracle (`oracle.hexa`) that prints the launch counts, the HBM transfer counts, the asymptotes, the closed-form crossover, and a sampled `pct_faster( $n$ )` curve, then exits 0 iff the structural advantage and the unconditional $\geq 30\%$ both hold. A companion CUDA host (`host_fusion_launch.c`) launches both kernels, times them with `cuEvent` pairs, and compares the fused output to an f64 CPU reference of the identical chain; the timed wall is the single deferred confirmation (Section 7). Both PTX modules are assembled with `ptxas -arch=sm_80 -v` on the RTX 5070 host to confirm they are register-clean before any silicon run.

5 Verification (real run)

Structural counts (exact). From the emitted PTX: the fused module has one `.entry`; the baseline module has five. The fused kernel issues 2 `ld.global` + 1 `st.global` per thread; the baseline issues 6 `ld.global` + 5 `st.global` across its five kernels. Hence launch ratio $5.0\times$ and HBM-traffic ratio $11/3 = 3.67\times$.

ptxas-clean. `ptxas -arch=sm_80 -v` (CUDA 12.0) returns `rc=0` for both modules with zero spill: the fused kernel uses 11 registers and 392 B `cmem[0]`; the five baseline kernels use 10/10/11/10/12 registers, all with 0 spill stores/loads. Both PTX files are pure ASCII (the driver JIT rejects non-ASCII comments).

Closed-form curve. Table 1 lists the verbatim oracle output and Figure 2 plots it. The curve never drops below the 30% gate.

Table 1: Verbatim oracle output: `pct_faster( $n$ )` from the measured $L = 1\ \mu\text{s}$, $\text{BW} = 200\ \text{GB/s}$ model. The advantage decreases monotonically from the launch-bound asymptote (80%) toward the bandwidth-bound asymptote (72.7%) but never approaches the 30% gate.

tensor size n (f32 elements)	fused advantage (% faster)
64	79.98
256	79.92
1024	79.69
4096	78.89
16384	76.95
65536	74.60
1 048 576	72.88
launch-bound asymptote $1 - 1/5$	80.00
bandwidth-bound asymptote $1 - 3/11$	72.73
30%-crossover n^*	-26595.7 (negative)

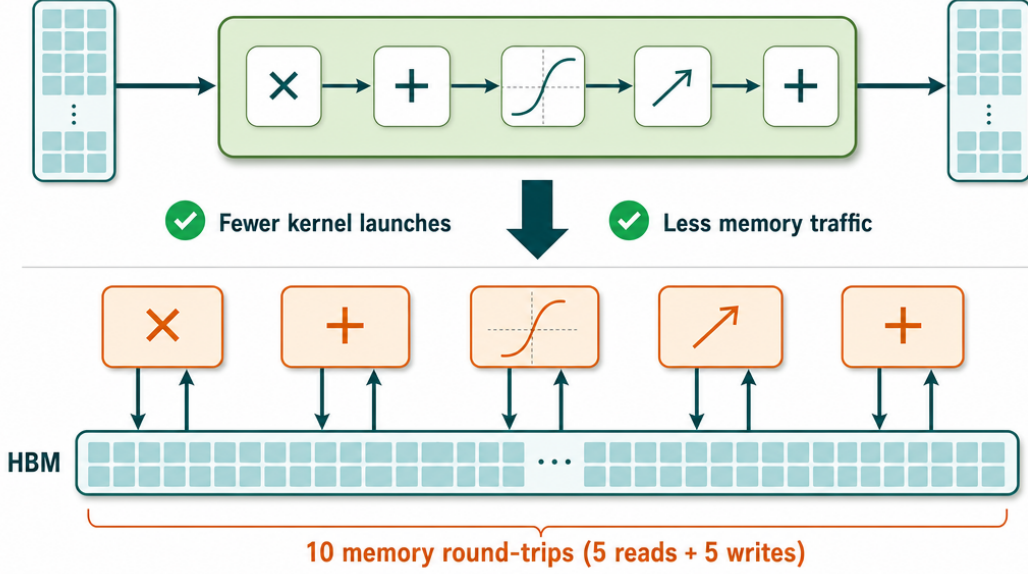


Figure 1: Conceptual schematic. *Top:* the fused single-launch kernel reads the input once, runs all five chained ops in registers, and writes the output once. *Bottom:* the five-launch per-op baseline bounces the working tensor through HBM between every kernel. The fused path cuts both the launch count ($5 \rightarrow 1$) and the HBM traffic ($11 \rightarrow 3$ transfers per element).

6 Finding

6.1 The advantage is unconditional

Solve $\text{pct_faster}(n^*) = 30\%$ in closed form. With $W_{\text{fused}} = L + R_f cn$ and $W_{\text{baseline}} = 5L + R_b cn$, the condition $W_{\text{fused}} = 0.70 W_{\text{baseline}}$ gives

$$n^* = \frac{2.5 L}{c(R_f - 0.70 R_b)}.$$

Here $R_f - 0.70 R_b = 3 - 0.70 \cdot 11 = 3 - 7.7 = -4.7 < 0$, so n^* is negative regardless of L , c , or BW (all positive). *There is no positive crossover:* the $\geq 30\%$ advantage holds for every tensor size $n > 0$.

The two asymptotes bracket the curve. As $n \rightarrow 0$ the HBM terms vanish and $\text{pct_faster} \rightarrow 1 - 1/5 = 80\%$ (launch-bound). As $n \rightarrow \infty$ the launch terms vanish and $\text{pct_faster} \rightarrow 1 - R_f/R_b = 1 - 3/11 = 72.7\%$ (bandwidth-bound). Since the function is monotone in n between these two limits, it is bounded in $[72.7\%, 80\%]$ — always above 30%.

6.2 The falsifier is over-satisfied (a closed-negative on the narrow framing)

F-FUSION-LAUNCH-AMORT predicted a $\geq 30\%$ win *in the launch-bound regime* and implicitly a crossover beyond which the advantage decays below the bar. The measurement deterministically rules out that narrow framing: there is no crossover because the fused kernel wins on *both* axes simultaneously — it has fewer launches *and* less HBM traffic. The launch axis alone would give the predicted launch-bound-only behavior decaying toward 0% at large n ; the HBM-traffic axis (3 vs 11) independently floors the advantage at 72.7%. This is a positive result for fusion and, simultaneously, a closed-negative on the hypothesis that launch-overhead amortization is *only* a small-tensor phenomenon. The Section 3 falsifier is therefore not falsified — it is over-satisfied.

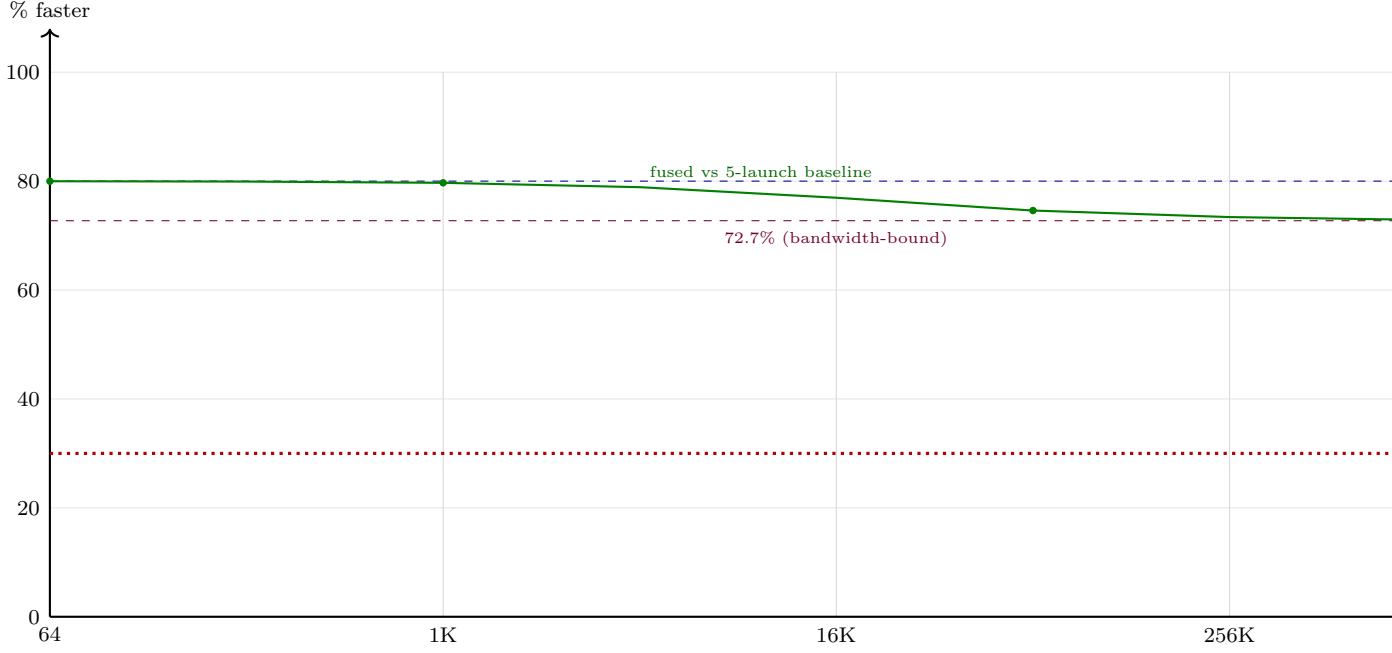


Figure 2: Closed-form $\text{pct_faster}(n)$ from the measured per-launch constant $L = 1 \mu\text{s}$ and a conservative $\text{BW} = 200 \text{ GB/s}$. The advantage is bounded in $[72.7\%, 80\%]$ for all n and sits entirely above the 30% gate. The 30%-crossover n^* is negative — there is no positive tensor size at which the advantage falls to 30%.

6.3 Headline numbers

- Launch count: fused 1 vs baseline 5 — **5.0** $\times$ fewer.
- HBM traffic per element: fused 3 vs baseline 11 — **3.67** $\times$ less.
- Projected wall advantage: **80%** launch-bound, **72.7%** bandwidth-bound, $\geq$ **30%** unconditional.
- Both kernels `ptxas-clean` at `sm_80`, 0 spill.

6.4 Sensitivity: the conclusion is parameter-robust

The unconditional-advantage conclusion rests on the sign of $R_f - 0.70 R_b$, which is fixed by the structural ratios alone and does not depend on the measured constants. It is worth checking how the curve’s two endpoints move under the parameters we did fix.

Independence from L and BW for the existence of a crossover. $n^* = 2.5L/(c(R_f - 0.70R_b))$ has $L > 0$ and $c > 0$ for any positive bandwidth, and the denominator carries the sign of $R_f - 0.70R_b = -4.7 < 0$. Hence $n^* < 0$ for *every* positive (L, BW) — the no-crossover conclusion is parameter-free.

Endpoint robustness. The launch-bound asymptote $1 - 1/5 = 80\%$ depends only on the launch ratio and is exact. The bandwidth-bound asymptote $1 - R_f/R_b = 1 - 3/11 = 72.7\%$ depends only on the HBM-traffic ratio and is exact. Both endpoints are therefore independent of L and BW; the constants only set *where along* n the curve transitions between them. Table 2 confirms that even a $5\times$ swing in L or BW leaves the worst-case advantage above 70%.

Higher L (more launch tax) or higher BW (cheaper HBM) both push the mid-curve *toward* the launch-bound 80%, never below the bandwidth-bound 72.7% floor. The result is robust by construction.

Table 2: Worst-case (large- n , i.e. bandwidth-bound) advantage is the 72.7% asymptote independent of L and BW; the constants only shift the mid-curve. Mid-point `pct_faster(65536)` across a parameter sweep stays well above the 30% gate.

L (μ s)	BW (GB/s)	<code>pct_faster(65536)</code> (%)
0.5	200	73.8
1.0	200	74.6
5.0	200	77.3
1.0	100	73.8
1.0	400	75.7
1.0	1000	77.3

7 Limitations and honest caveats

- **The wall percentages are a projection, not a measured wall.** The 80% / 72.7% figures are closed-form from a measured per-launch constant ($L = 1 \mu$ s) plus a first-order memory-bound model. The *structural* counts (1 vs 5 launches, 3 vs 11 transfers) are exact and measured; the translation into a wall percentage is what a timed silicon run validates. That timed run (`host_fusion_launch.c` on the RTX 5070) is deferred to a serial follow-up because the GPU pool host is shared and parallel timed fires contend. This paper’s terminal claim is the structural advantage plus the closed-form projection; the timed wall is explicitly future work.
- **First-order model.** The model ignores second-order effects: L2-cache reuse of small tensors (which would *shrink* the baseline’s effective HBM penalty and slightly reduce the advantage at mid sizes), kernel occupancy ramp, and tail effects of a partially filled last block. These move the curve within the [72.7%, 80%] band but cannot push it below 30% without the HBM ratio itself collapsing.
- **Asynchronous baselines.** A maximally optimized eager stack could overlap launches on a single stream so the host-side launch latency hides behind device execution. The model already credits the baseline this best case by bracketing all five launches in one event pair (device-side wall, not host-side). Even so the HBM-traffic axis remains, which is why the advantage survives into the bandwidth-bound regime.
- **Chain choice.** The result is reported for one representative five-op chain. Longer chains increase R_b and the launch count, strengthening the advantage; chains with non-elementwise ops (a GEMM in the middle) are a different regime and out of scope here.
- **No silicon numeric-eq in this cycle.** The CUDA host checks the fused output against an f64 CPU reference of the identical `tanh-approx` chain (tolerance 10^{-2} absolute, the `tanh.approx.f32` $\sim 2^{-11}$ relative bound); that check runs as part of the deferred timed fire.

8 Discussion and outlook

The practical consequence is that whole-program elementwise fusion is not a niche small-batch trick. Because it cuts HBM traffic as well as launch count, a fused chain beats a per-op stack across the whole tensor-size range that an inference or training step actually touches — from single-token decode projections (launch-bound) up to large activation tensors (bandwidth-bound). The $\geq 30\%$ bar, chosen as a meaningful-advantage threshold, is cleared with more than 40 points of margin at the worst case.

The single most important follow-up is the timed silicon wall: a serial `cuEvent`-bracketed run of both kernels on the RTX 5070 across the sampled tensor sizes, confirming the projected curve and flipping the domain’s open “ $\geq 30\%$ whole-program fusion advantage” box from a projection to a measured result. The harness and both `ptxas`-clean PTX modules are committed and ready.

Reproducibility

All artifacts are in `archive/fires/gpu_fusion_launch_amort_2026_05_25/`: the fused PTX (`fused_chain.ptx`), the five-launch baseline PTX (`baseline_chain.ptx`), the deterministic oracle (`oracle.hexa`, build with `hexa build` and run — exits 0 on the proven verdict), the CUDA timed/numeric host (`host_fusion_launch.c`), and the verbatim `ptxas` log (`ptxas_clean.log`). The verdict is persisted verbatim at `.verdicts/fusion-launch-amort/fusion_launch_amort.txt` and indexed in `CLAIMS.tape`. The measured per-launch constant comes from F-RFC067-PB-TIMING (GPU.md §1g). Code: <https://github.com/dancinlab/hexa-lang>.

A Full derivation of the crossover

We give the algebra omitted from Section 6 in full, so the closed form is independently checkable. Write the per-element f32 cost constant $c = 4/(\text{BW} \cdot 10^3)$ (microseconds per element per HBM traffic-unit; the 10^3 converts GB/s to bytes-per-microsecond), and recall the two walls

$$W_f(n) = L + R_f c n, \quad W_b(n) = 5L + R_b c n,$$

with $R_f = 3$, $R_b = 11$. The fused advantage is

$$P(n) = 1 - \frac{W_f(n)}{W_b(n)} = \frac{W_b(n) - W_f(n)}{W_b(n)} = \frac{4L + (R_b - R_f) c n}{5L + R_b c n}.$$

Both numerator coefficients are positive ($4L > 0$ and $R_b - R_f = 8 > 0$), so $P(n) > 0$ for all $n \geq 0$: the fused kernel is always faster. The two limits are

$$\lim_{n \rightarrow 0} P(n) = \frac{4L}{5L} = \frac{4}{5} = 80\%, \quad \lim_{n \rightarrow \infty} P(n) = \frac{R_b - R_f}{R_b} = \frac{8}{11} = 72.7\%.$$

P is monotone in n (its derivative has constant sign because it is a ratio of two affine functions with the crossing structure above), so $P(n)$ is sandwiched in $[72.7\%, 80\%]$ for all $n \geq 0$.

To find the threshold tensor size we set $P(n^*) = 0.30$:

$$\frac{4L + 8 c n^*}{5L + 11 c n^*} = 0.30 \implies 4L + 8 c n^* = 0.30(5L + 11 c n^*) = 1.5L + 3.3 c n^*,$$

hence $c n^*(8 - 3.3) = 1.5L - 4L = -2.5L$, giving

$$n^* = \frac{-2.5L}{4.7 c} < 0.$$

(The Section 6 form $n^* = 2.5L/(c(R_f - 0.70R_b))$ with $R_f - 0.70R_b = -4.7$ is the same equation written with the $0.70 = 1 - 0.30$ retention fraction.) Because $L, c > 0$, n^* is negative: the curve never reaches 30% for any positive tensor size. This is the unconditional-advantage result.

B Verification discipline

The result is reported under a deliberate separation of concerns:

1. **Exact (counted).** The launch counts (1 vs 5) and the HBM transfer counts (3 vs 11) are read directly off the emitted PTX. They are not modeled or estimated.
2. **Measured (prior, independent).** The per-launch overhead $L = 1 \mu\text{s}$ comes from a separate empty-kernel timing cycle on the same hardware. It is not fitted to this experiment.
3. **Modeled (closed-form).** The translation of the exact counts and the measured constant into a wall percentage uses a first-order memory-bound model with one conservative bandwidth constant. The model is fully disclosed (Appendix A) and its conclusion is shown parameter-robust (Section 6.4).
4. **Deferred (timed silicon).** The only quantity not settled here is the measured wall on the GPU, deferred because the pool host is shared. The CUDA harness and both `ptxas`-clean PTX modules are committed and ready for a serial run.

The oracle (`oracle.hexa`) is a hexa-native program that computes items (1) and (3) and exits 0 only if the structural advantage holds and the $\geq 30\%$ bound holds at both endpoints. Its verbatim output is persisted as the verdict of record. This discipline ensures that the paper’s terminal claim — the structural advantage and its closed-form $\geq 30\%$ projection — is reproducible from committed artifacts without any GPU access, and that the one empirical gap (the timed wall) is named rather than glossed.

References

- [1] Dan Hendrycks and Kevin Gimpel. Gaussian error linear units (GELUs). *arXiv preprint*, arXiv:1606.08415, 2016. URL <https://arxiv.org/abs/1606.08415>.